

Lab 8

Automated Loan Management System with Email Notification Using Stored Procedures

Introduction:

In this lab, we focus on advanced SQL concepts including stored procedures, automated data processing, and relational database management for a loan management system. Stored procedures are precompiled SQL statements that can be executed multiple times, providing better performance, security, and code reusability. This lab demonstrates the creation of a stored procedure that automatically generates email notifications for customers with overdue loan payments.

The lab covers:

- **CREATE PROCEDURE** – Creating stored procedures for business logic automation
- **Date Functions** – Using GETDATE () for current date operations
- **Window Functions** – Using ROW_NUMBER () for sequential numbering
- **JOIN Operations** – Combining data from multiple related tables
- **Conditional Data Insertion** – Inserting data based on specific criteria
- **Foreign Key Constraints** – Maintaining referential integrity in procedural contexts

Step 1: Creating the Database Tables

First, we establish the foundation by creating four interconnected tables: customerdetails, loantype, loandetails, and emaildetails.

SQL Code:

```
-- 1 Drop existing tables if they exist
DROP TABLE IF EXISTS emaildetails;
DROP TABLE IF EXISTS loandetails;
DROP TABLE IF EXISTS loantype;
DROP TABLE IF EXISTS customerdetails;

-- 2 Create customerdetails table
CREATE TABLE customerdetails (
    cid INT NOT NULL PRIMARY KEY,
    customername VARCHAR(100),
    email VARCHAR(100),
    phoneno BIGINT,
    address VARCHAR(100)
);

-- 3 Create loantype table
```

```
CREATE TABLE loantype (
    sno INT NOT NULL PRIMARY KEY,
    loantype CHAR(50) UNIQUE -- unique so it can be referenced
);
```

-- 4 Create loandetails table with EMI tracking

```
CREATE TABLE loandetails (
    sno INT NOT NULL PRIMARY KEY,
    cid INT, -- link to customer
    loanname VARCHAR(100),
    loantype CHAR(50),
    interest INT,
    due_date DATE,
    paid_date DATE NULL, -- NULL if unpaid
    FOREIGN KEY (loantype) REFERENCES loantype(loantype),
    FOREIGN KEY (cid) REFERENCES customerdetails(cid)
);
```

-- 5 Create emaildetails table

```
CREATE TABLE emaildetails (
    sno INT NOT NULL,
    [From] VARCHAR(100),
    [To] VARCHAR(100),
    cc VARCHAR(100),
    bcc VARCHAR(100),
    emailbody VARCHAR(100),
    status VARCHAR(100)
);
```

Step 2: Inserting Sample Data

-- Insert sample customers

```
INSERT INTO customerdetails (cid, customername, email, phoneno, address)
VALUES
(1, 'Sanjog Gautam', 'Sanjog@gmail.com', 9876543210, 'New York'),
(2, 'Sarfaraz Alam', 'Sarfaraz Alam@gmial.com', 9123456780, 'Los Angeles'),
(3, 'Susan Chaudhary', 'Susan Chaudhary@gmail.com', 9988776655, 'Chicago');
```

-- Insert loan types

```
INSERT INTO loantype (sno, loantype)
VALUES
(1, 'Home Loan'),
(2, 'Car Loan'),
(3, 'Personal Loan');
```

-- Insert loans with EMI info

```
INSERT INTO loandetails (sno, cid, loanname, loantype, interest, due_date,
paid_date)
VALUES
(1, 1, 'Dream Home Plan', 'Home Loan', 7, '2025-08-01', NULL), -- overdue
(2, 2, 'Swift Car Plan', 'Car Loan', 9, '2025-08-15', '2025-08-10'), -- paid

(3, 3, 'Holiday Special', 'Personal Loan', 12, '2025-08-05', NULL); -- overdue
```

Step 3: Creating the Stored Procedure

We create a stored procedure named 'xyz' that automatically generates email notifications for overdue loans.

```
CREATE PROCEDURE xyz
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @today DATE = GETDATE();

    INSERT INTO emaildetails (sno, [From], [To], cc, bcc, emailbody, status)
    SELECT
        ISNULL((SELECT MAX(sno) FROM emaildetails), 0)
        + ROW_NUMBER() OVER (ORDER BY c.cid) AS sno,
        'bank@example.com' AS [From],
        c.email AS [To],
        NULL AS cc,
        NULL AS bcc,
        CONCAT('Dear ', c.customername, ', your EMI for loan "', l.loaname, '" is
        overdue. Please pay immediately.') AS emailbody,
        'Pending' AS status
    FROM loandetails l
    JOIN customerdetails c ON l.cid = c.cid
    WHERE l.paid_date IS NULL
        AND l.due_date < @today;
END;

GO
```

Step 4: Executing the Stored Procedure

```
EXEC xyz;

SELECT * FROM emaildetails;
```

Output:

Results		Messages					
	sno	From	To	cc	bcc	emailbody	status
1	1	bank@example.com	Sanjog@gmail.com	NULL	NULL	Dear Sanjog Gautam, your EMI for loan "Dream Ho...	Pending
2	2	bank@example.com	Susan Chaudhary@gmail.com	NULL	NULL	Dear Susan Chaudhary, your EMI for loan "Holiday S...	Pending

Fig 7.1: The emaildetails table shows two automated email notifications generated for customers with overdue loans. Sanjog Gautam has an overdue Home Loan "Dream Home Plan" and Susan Chaudhary has an overdue Personal Loan "Holiday Special". The customer with paid loan (Sarfaraz Alam) did not receive any notification, demonstrating the procedure's conditional logic.

Conclusion:

This lab successfully demonstrated the creation and execution of stored procedures that automate business processes while maintaining referential integrity. The procedure 'xyz' effectively:

- **Identifies overdue loans** by comparing due dates with current date
- **Generates personalized emails** using customer and loan information
- **Automates notification process** reducing manual effort and human error
- **Maintains data consistency** through proper foreign key relationships
- **Provides scalable solution** for handling multiple customers and loan types.

Lab 9

Parameterized Stored Procedures with CRUD Operations

Introduction:

In this lab, we create a stored procedure that can perform multiple operations (SELECT and INSERT) on a database table using parameters. Instead of writing separate procedures for each operation, we use one procedure with a flag parameter to control what action to perform. This approach is commonly used in real-world applications for efficient database management.

The lab covers:

- **Parameterized Procedures** – Creating procedures with input parameters
- **Flag-Based Operations** – Using a flag to select different operations
- **Input Validation** – Checking if required data is provided
- **Error Handling** – Using TRY-CATCH to handle errors gracefully

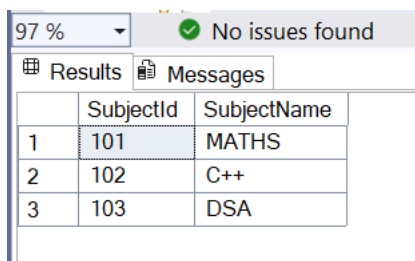
Step 1: Creating the SubjectDetails Table

```
-- Drop existing table if it exists
DROP TABLE IF EXISTS SubjectDetails;

-- Create SubjectDetails table
CREATE TABLE SubjectDetails (
    SubjectId INT NOT NULL PRIMARY KEY,
    SubjectName VARCHAR(100)
);

-- Insert sample data
INSERT INTO SubjectDetails (SubjectId, SubjectName) VALUES
(101, 'MATHS'),
(102, 'C++'),
(103, 'DSA');

-- Verify the data
SELECT * FROM SubjectDetails;
```



The screenshot shows a SQL Server interface with a status bar indicating '97 %' and 'No issues found'. Below the status bar, there are tabs for 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with the following data:

	SubjectId	SubjectName
1	101	MATHS
2	102	C++
3	103	DSA

Fig 8.1: The SubjectDetails table is created with 3 sample records.

Step 2: Creating the Stored Procedure

We create a procedure called 'sp_SubjectDetails' that can do two things:

1. **SELECT:** Retrieve data from the table (when @flag = 's')
2. **INSERT:** Add new data to the table (when @flag = 'i')

```
Alter Procedure sp_SubjectDetails
(
    @flag Char,
    @SubjectId Int = NULL,
    @SubjectName Varchar(100) = NULL
)
AS
BEGIN
    If @flag = 's' -- SELECT Data From Table
    BEGIN
        IF @SubjectId IS NULL
        BEGIN
            Select '0' As STATUS_CODE,
                'Data Retrived From Table' AS STATUS_MSG
            Select
                SubjectId,
                SubjectName
            From SubjectDetails
            RETURN
        END
        Select
            SubjectId,
            SubjectName
        From SubjectDetails WHERE SubjectId = @SubjectId
    END

    If @flag = 'i' -- INSERT DATA INTO TABLE
    BEGIN
        IF @SubjectId IS NULL OR @SubjectId = ''
        BEGIN
            Select '100' As STATUS_CODE,
                'Subject ID is Missing' AS STATUS_MSG
            RETURN
        END

        IF @SubjectName IS NULL OR @SubjectName = ''
        BEGIN
            Select '101' As STATUS_CODE,
                'Subject Name is Missing' AS STATUS_MSG
            RETURN
        END
    END

    BEGIN TRY
        Insert Into SubjectDetails(SubjectId, SubjectName)
        Values(@SubjectId, @SubjectName)

        Select '0' AS STATUS_CODE,
```

```

        'New Subject is Added successfully' AS STATUS_MSG

    END TRY

    BEGIN CATCH
        Select ERROR_NUMBER() AS STATUS_CODE,
               ERROR_MESSAGE() AS STATUS_MSG
    END CATCH

END
END

```

How It Works:

Parameters:

- @flag: Tells the procedure what to do ('s' = SELECT, 'i' = INSERT)
- @SubjectId: The subject ID number
- @SubjectName: The subject name

For SELECT (@flag = 's'):

- If no SubjectId is given, it shows all subjects
- If SubjectId is given, it shows only that subject

For INSERT (@flag = 'i'):

- First checks if SubjectId is provided (if not, returns error code 100)
- Then checks if SubjectName is provided (if not, returns error code 101)
- If both are provided, it inserts the new subject
- Uses TRY-CATCH to handle any database errors

Step 3: Retrieving All Data (SELECT Operation)

We execute the procedure with flag 's' to get all subjects.

sp_help

```
EXEC sp_SubjectDetails @flag='s', @SubjectId = NULL, @SubjectName = NULL;
```

97 %	1	0	↑	↓	←
Results Messages					
	Name	Owner	Object_type		
1	SubjectDetails	dbo	user table		
2	sp_SubjectDetails	dbo	stored procedure		
3	EventNotificationErrorsQueue	dbo	queue		
4	QueryNotificationErrorsQueue	dbo	queue		
5	ServiceBrokerQueue	dbo	queue		
6	PK_SubjectD__AC1BA3A85674012B	dbo	primary key cns		
7	queue_messages_1977058079	dbo	internal table		
8	queue_messages_2009058193	dbo	internal table		
	User_type	Storage_type	Length	Prec	Scale
	Nullable	Default_name	Rule_name	Collation	
	STATUS_CODE	STATUS_MSG			
1	0	Data Retrived From Table			
	SubjectId	SubjectName			
1	101	MATHS			
2	102	C++			
3	103	DSA			

Fig 8.2: The procedure successfully retrieves all subjects from the table. Status code '0' means the operation was successful.

Step 4: Retrieving Specific Data

We can also get a specific subject by providing its ID.

```
EXEC sp_SubjectDetails @flag='s', @SubjectId = 102, @SubjectName = NULL;
```

Results Messages		
	SubjectId	SubjectName
1	102	C++

Fig 8.3: The procedure returns only the subject with ID 102.

Step 5: Inserting New Data (INSERT Operation)

Now we use the same procedure to add a new subject to the table.

```
EXEC sp_SubjectDetails @flag='i',
    @SubjectId = 104,
    @SubjectName = 'Compiler Construction and Design';

-- Verify the insertion

SELECT * FROM SubjectDetails;
```


Results		Messages
	STATUS_CODE	STATUS_MSG
1	0	New Subject is Added successfully

	SubjectId	SubjectName
1	101	MATHS
2	102	C++
3	103	DSA
4	104	Compiler Construction and Design

Fig 8.4: The new subject "Compiler Construction and Design" with ID 104 is successfully added to the table.

Step 6: Testing Error Handling

Let's see what happens if we try to insert without providing the SubjectId.

```
EXEC sp_SubjectDetails @flag='i',
    @SubjectId = NULL,

    @SubjectName = 'Database Management';
```

Results		Messages
	STATUS_CODE	STATUS_MSG
1	100	Subject ID is Missing

Fig 8.5: The procedure checks for missing data and returns an error message instead of crashing.

Step 7: Using sp_help Command

We can use the built-in sp_help command to see information about our table.

```
sp_help
```

```
exec sp_help SubjectDetails
```

97 %	✓ No issues found	Ln: 4	Ch: 1
Results Messages			
	Name	Owner	Object_type
1	SubjectDetails	dbo	user table
2	sp_SubjectDetails	dbo	stored procedure
3	EventNotificationErrorsQueue	dbo	queue
4	QueryNotificationErrorsQueue	dbo	queue
5	ServiceBrokerQueue	dbo	queue
6	PK__SubjectD__AC1BA3A85674012B	dbo	primary key cns
7	queue_messages_1977058079	dbo	internal table
8	queue_messages_2009058193	dbo	internal table
	User_type	Storage_type	Length
	Prec	Scale	Nullable
	Default_name	Rule_name	Collation
	Name	Owner	Type
	Created_datetime		
1	SubjectDetails	dbo	user table
	2025-12-06 19:02:25.913		
	Column_name	Type	Computed
	Length	Prec	Scale
	Nullable	TrimTrailingBlanks	FixedLenNullInSource
	Collation		
1	SubjectId	int	no
	4	10	0
	no	no	(n/a)
	(n/a)	yes	yes
	SQL_Latin1_General_CP1_CI_AS		
	Identity	Seed	Increment
	Not For Replication		
1	No identity column defined.	NULL	NULL
	NULL		
	RowGuidCol		
1	No rowguidcol column defined.		
	Data_located_on_filegroup		
1	PRIMARY		
✓ Query executed successfully.			
(local) (16.0 RTM) GT\gauta (72) lab 8 00:00:			

Fig 8.6: The sp_help command shows detailed information about the SubjectDetails table including column names, data types, and constraints.

Conclusion:

This lab successfully demonstrated how to create a parameterized stored procedure that handles multiple operations. The procedure 'sp_SubjectDetails' effectively:

- **Performs SELECT operations** to retrieve all or specific subjects
- **Performs INSERT operations** to add new subjects to the table
- **Validates input data** before processing to ensure data quality
- **Handles errors gracefully** using TRY-CATCH blocks
- **Returns status codes** to indicate success or failure

This approach is practical and efficient for real-world applications where we need to perform multiple database operations through a single, reusable procedure.

Lab 10

Transaction Management in Banking System Using BEGIN TRANSACTION, COMMIT, and ROLLBACK

Introduction:

In this lab, we implement a banking transaction system that demonstrates the critical concept of database transactions. A transaction is a set of SQL operations that must be executed as a single unit - either all operations succeed together, or all fail together. This is essential in banking systems where money transfer between accounts must be atomic (all-or-nothing) to maintain data consistency and prevent financial errors.

The lab covers:

- **Transaction Control** – Using BEGIN TRANSACTION, COMMIT, and ROLLBACK
- **ACID Properties** – Ensuring Atomicity, Consistency, Isolation, and Durability
- **Balance Validation** – Preventing negative account balances
- **Error Handling** – Rolling back transactions when conditions fail
- **Stored Procedures** – Encapsulating transaction logic for reusability

Scenario: Transfer money from Account A (source) to Account B (destination). If Account A has insufficient balance, the entire transaction should be cancelled.

Step 1: Creating the Database and Table

First, we create a new database and the Accounts table to store customer account information.

```
-- Create a new database
CREATE DATABASE txn;
GO

-- Use the database
USE txn;
GO

-- Create the Accounts table
CREATE TABLE Accounts (
    AccountID INT PRIMARY KEY,
    Balance DECIMAL(10, 2) NOT NULL
);

-- Insert sample data
INSERT INTO Accounts (AccountID, Balance)
```

```
VALUES (1, 500.00), (2, 300.00);
```

```
-- Verify the data
```

```
SELECT * FROM Accounts;
```

Results		Messages	
	AccountID	Balance	
1	1	500.00	
2	2	300.00	

Fig 9.1: The Accounts table is created with two accounts. Account 1 has a balance of 500.00 and Account 2 has a balance of 300.00.

Step 2: Creating the Transaction Stored Procedure

We create a stored procedure that handles money transfer between two accounts using transaction control.

```
CREATE PROCEDURE TransferAmount
    @fromAccount INT,
    @toAccount INT,
    @transferAmount DECIMAL(10, 2)
AS
BEGIN
    DECLARE @balanceA DECIMAL(10, 2);

    -- Start the transaction
    BEGIN TRANSACTION;

    BEGIN TRY
        -- Check the balance of the source account
        SELECT @balanceA = Balance
        FROM Accounts
        WHERE AccountID = @fromAccount;

        -- Perform the transaction if balance is sufficient
        IF @balanceA >= @transferAmount
        BEGIN
            -- Debit from source account
            UPDATE Accounts
            SET Balance = Balance - @transferAmount
            WHERE AccountID = @fromAccount;

            -- Credit to target account
            UPDATE Accounts
            SET Balance = Balance + @transferAmount
            WHERE AccountID = @toAccount;

            -- Commit the transaction
            COMMIT TRANSACTION;
            SELECT 'Transaction completed successfully.' AS Message;
        END
    ELSE
```

```

        BEGIN
            -- Rollback the transaction if insufficient balance
            ROLLBACK TRANSACTION;
            SELECT 'Transaction failed: Insufficient balance in the source account.'
AS Message;
        END
    END TRY
    BEGIN CATCH
        -- Rollback in case of any error
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        SELECT ERROR_MESSAGE() AS Message;
    END CATCH
END;

GO

```

How the Procedure Works:

1. Parameters:

- @fromAccount: Source account ID (where money is debited)
- @toAccount: Destination account ID (where money is credited)
- @transferAmount: Amount to transfer

Note: SQL Server uses @ prefix for parameters, unlike MySQL which uses IN keyword.

2. Transaction Steps:

- **BEGIN TRANSACTION:** Begins a new transaction
- **SELECT @balanceA = Balance:** Checks the current balance of source account
- **IF @balanceA >= @transferAmount:** Validates if transfer is possible
- **UPDATE (Debit):** Subtracts amount from source account
- **UPDATE (Credit):** Adds amount to destination account
- **COMMIT TRANSACTION:** Makes all changes permanent if successful
- **ROLLBACK TRANSACTION:** Cancels all changes if validation fails
- **TRY-CATCH:** Handles any unexpected errors during the transaction

Step 3: Testing Successful Transaction

We test the procedure by transferring 200.00 from Account 1 to Account 2.

```

-- Transfer 200.00 from Account 1 to Account 2
EXEC TransferAmount @fromAccount = 1, @toAccount = 2, @transferAmount = 200.00;

-- Check the updated balances

SELECT * FROM Accounts;

```

Results Messages	
Message	
1	Transaction completed successfully.

	AccountID	Balance
1	1	300.00
2	2	500.00

Fig 9.2: The transaction is successful. Account 1 balance decreased from 500.00 to 300.00 (debited 200.00), and Account 2 balance increased from 300.00 to 500.00 (credited 200.00). The transaction was committed successfully.

Step 4: Testing Failed Transaction (Insufficient Balance)

Now we test what happens when trying to transfer more money than available in the source account.

```
-- Attempt to transfer 400.00 from Account 1 (which only has 300.00)
EXEC TransferAmount @fromAccount = 1, @toAccount = 2, @transferAmount = 400.00;

-- Check the balances (should remain unchanged)

SELECT * FROM Accounts;
```

Results Messages	
Message	
1	Transaction failed: Insufficient balance in the ...

	AccountID	Balance
1	1	300.00
2	2	500.00

Fig 9.3: The transaction fails because Account 1 only has 300.00 but the transfer amount is 400.00. The ROLLBACK command cancels any changes, so both account balances remain unchanged. This demonstrates transaction atomicity.

Step 5: Testing Edge Case (Exact Balance Transfer)

Let's test transferring the exact balance available in an account.

```
-- Transfer exactly 300.00 from Account 1 (empties the account)
EXEC TransferAmount @fromAccount = 1, @toAccount = 2, @transferAmount = 300.00;

-- Check the updated balances

SELECT * FROM Accounts;
```

Results		Messages	
		Message	
1		Transaction completed successfully.	

	AccountID	Balance
1	1	0.00
2	2	800.00

Fig 9.4: The transaction successfully transfers all 300.00 from Account 1 to Account 2. Account 1 now has zero balance, which is acceptable (not negative). This shows the procedure handles exact balance transfers correctly.

Conclusion:

This lab successfully demonstrated transaction management in a banking system using stored procedures with transaction control commands. The TransferAmount procedure effectively:

- **Ensures atomicity** by using START TRANSACTION, COMMIT, and ROLLBACK
- **Validates business rules** by checking sufficient balance before transfer
- **Prevents data inconsistency** by rolling back failed transactions
- **Maintains data integrity** by preventing negative account balances
- **Provides clear feedback** through success/failure messages

The implementation showcases essential database concepts that are critical for any financial or business-critical application. Understanding transaction management is fundamental for building reliable systems that handle sensitive data and operations where partial completion is not acceptable. This approach ensures that the database always remains in a consistent state, even when operations fail.

Lab 11

Cartesian Product and JOIN Operations

Introduction:

In this lab, we explore the concept of Cartesian Product in database systems. A Cartesian Product combines every row from one table with every row from another table. We also learn how to filter this product to create meaningful relationships between students and their courses based on their academic stream.

The lab covers:

- **Cartesian Product** – All possible combinations between tables
- **Filtered Cartesian Product** – Using conditions to get relevant data
- **JOIN Operations** – Combining tables efficiently
- **Views** – Creating simplified data representations

Step 1: Creating Tables and Inserting Data

We create four tables to manage student course enrollment and populate them with sample data.

```
-- Create tables
CREATE TABLE Students (
    std_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    stream VARCHAR(50)
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50)
);

CREATE TABLE StudentCourses (
    std_id INT,
    course_id INT,
    semester INT,
    PRIMARY KEY (std_id, course_id, semester),
    FOREIGN KEY (std_id) REFERENCES Students(std_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

CREATE TABLE StudentSemester (
    std_id INT,
    semester INT,
```



```

        PRIMARY KEY (std_id, semester),
        FOREIGN KEY (std_id) REFERENCES Students(std_id)
    );

-- Insert sample data
INSERT INTO Students (std_id, student_name, streamm)
VALUES
(1, 'Student A', 'Science'),
(2, 'Student B', 'Humanities'),
(3, 'Student C', 'Science'),
(4, 'Student D', 'Humanities');

INSERT INTO Courses (course_id, course_name)
VALUES
(1, 'NM'),
(2, 'CA'),
(3, 'DSA'),
(4, 'History'),
(5, 'Sociology'),
(6, 'Psychology');

INSERT INTO StudentSemester (std_id, semester)
VALUES
(1, 3),
(2, 3),
(3, 2),
(4, 1);

-- Verify data
SELECT * FROM Students;
SELECT * FROM Courses;

SELECT * FROM StudentSemester;

```

96 % ✓ No issues found

Results Messages

	std_id	student_name	streamm
1	1	Student A	Science
2	2	Student B	Humanities
3	3	Student C	Science
4	4	Student D	Humanities

	course_id	course_name
1	1	NM
2	2	CA
3	3	DSA
4	4	History
5	5	Sociology
6	6	Psychology

	std_id	semester
1	1	3
2	2	3
3	3	2
4	4	1

Fig 10.1: The tables are created with 4 students (2 Science, 2 Humanities) and 6 courses (3 Science courses, 3 Humanities courses).

Step 2: Understanding Cartesian Product

A Cartesian Product creates all possible combinations between two tables.

-- Cartesian Product: All possible student-course combinations

```
SELECT
    s.student_name,
    s.streamm,
    c.course_name
FROM Students s, Courses c

ORDER BY s.std_id, c.course_id;
```

	student_name	streamm	course_name
1	Student A	Science	NM
2	Student A	Science	CA
3	Student A	Science	DSA
4	Student A	Science	History
5	Student A	Science	Sociology
6	Student A	Science	Psychology
7	Student B	Humanities	NM
8	Student B	Humanities	CA
9	Student B	Humanities	DSA
10	Student B	Humanities	History
11	Student B	Humanities	Sociology
12	Student B	Humanities	Psychology
13	Student C	Science	NM
14	Student C	Science	CA
15	Student C	Science	DSA
16	Student C	Science	History
17	Student C	Science	Sociology
18	Student C	Science	Psychology
19	Student D	Humanities	NM
20	Student D	Humanities	CA
21	Student D	Humanities	DSA
22	Student D	Humanities	History
23	Student D	Humanities	Sociology
24	Student D	Humanities	Psychology

Fig 10.2: The Cartesian Product produces 24 rows (4 students × 6 courses = 24). Notice that Science students are paired with Humanities courses and vice versa, which doesn't make sense academically.

Step 3: Creating Filtered Course Enrollment

Now we filter the Cartesian Product to enroll students only in courses matching their stream.

```
-- Insert filtered enrollments based on stream
INSERT INTO StudentCourses (std_id, course_id, semester)
SELECT s.std_id, c.course_id, ss.semester
FROM Students s
JOIN StudentSemester ss ON s.std_id = ss.std_id
JOIN Courses c ON (
    (s.streamm = 'Science' AND c.course_name IN ('NM', 'CA', 'DSA')) OR
    (s.streamm = 'Humanities' AND c.course_name IN ('History', 'Sociology',
'Psychology'))
);

-- Verify enrollments

SELECT * FROM StudentCourses ORDER BY std_id, course_id;
```

96 % 19 0

Results		Messages	
	std_id	course_id	semester
1	1	1	3
2	1	2	3
3	1	3	3
4	2	4	3
5	2	5	3
6	2	6	3
7	3	1	2
8	3	2	2
9	3	3	2
10	4	4	1
11	4	5	1
12	4	6	1

Fig 10.3: The filtered result shows only 12 rows (reduced from 24). Each Science student is enrolled in 3 Science courses, and each Humanities student is enrolled in 3 Humanities courses.

How it Works:

- **Start with:** All student-course combinations (24 rows)
- **Filter by:** Student stream matches course type
- **Result:** Only valid enrollments (12 rows)

Step 4: Creating a View for Easy Access

We create a view that combines all information for easy querying.

```
-- Create view
CREATE VIEW StudentCourseDetails AS
SELECT
    s.std_id,
    s.student_name,
    sc.semester,
    s.streamm,
    c.course_name
FROM
    Students s
JOIN
    StudentCourses sc ON s.std_id = sc.std_id
JOIN
    Courses c ON sc.course_id = c.course_id;
GO

-- Query the view

SELECT * FROM StudentCourseDetails ORDER BY std_id, course_name;
```

96 % 14 0

Results Messages

	std_id	student_name	semester	streamm	course_name
1	1	Student A	3	Science	CA
2	1	Student A	3	Science	DSA
3	1	Student A	3	Science	NM
4	2	Student B	3	Humanities	History
5	2	Student B	3	Humanities	Psychology
6	2	Student B	3	Humanities	Sociology
7	3	Student C	2	Science	CA
8	3	Student C	2	Science	DSA
9	3	Student C	2	Science	NM
10	4	Student D	1	Humanities	History
11	4	Student D	1	Humanities	Psychology
12	4	Student D	1	Humanities	Sociology

Query executed successfully

Fig 10.4: The view displays complete student enrollment information in an easy-to-read format, showing each student with their courses, stream, and semester.

Conclusion:

This lab demonstrated how Cartesian Product works and how to filter it for practical use. We learned that:

- **Cartesian Product** generates all possible combinations (24 rows from 4 students $\times$ 6 courses)
- **Filtering** reduces results to meaningful data (12 valid enrollments)
- **Views** simplify data access by combining multiple tables
- **Business rules** ensure data quality (students only get courses from their stream)

Understanding Cartesian Products helps us write better queries and understand how database joins work efficiently.